1 Travail à faire

L'objectif de ce TP est d'implémenter en Python des algorithmes de résolution du problème dual de Lagrange, et de les appliquer à différentes instances de test.

1.1 Méthodes à implémenter

Vous devez implémenter au minimum les méthodes suivantes, pour chacune des relaxations décrites dans la section :

1. Méthode de sous-gradient basique (section 3.3.2 du cours).
2. Méthode de sous-gradient avec *Average Direction Strategy* (ADS, section 3.3.3 du cours).

Le travail est à réaliser en binômes. Pour ce travail, **l'usage d'IA génératives est interdit** (par intérêt pédagogique, vous devez apprendre à concevoir et implémenter ces méthodes au moins une fois par vous-même).

Le devoir surveillé terminal comportera des questions concernant le développement théorique et l'implémentation des relaxations lagrangiennes de ce TP. La note de TP prendra en compte vos réponses à ces questions.

Paramétrage des méthodes
Les méthodes de sous-gradient utilisent différents paramètres pouvant influer sur le déroulement de la résolution. En particulier, il existe différentes stratégies pour déterminer la taille du pas à employer.

— Le choix de la stratégie de définition de la taille du pas est libre. Par défaut, vous pouvez implémenter la règle très simple suivante :
  1. Définir un pas initial $s = 2$.
  2. A chaque étape, décroître la valeur de s : $s \leftarrow \alpha s$, avec $0 < \alpha < 1$ (par exemple, $\alpha = 0.8$).
— L'implémentation correcte d'une stratégie plus élaborée donnera lieu à une note plus élevée.
— Le choix d'éventuels paramètres est laissé à votre discrétion. En particulier, le paramétrage des critères d'arrêt devra permettre d'obtenir une bonne solution approchée du problème dual.

Méthodes supplémentaires Afin d'améliorer votre note, vous pouvez implémenter, en plus, une ou des méthodes parmi les suivantes :

3. Méthode de plans coupants (section 3.4 du cours).
4. *Modified Deflected Subgradient* (MDS, section 3.3.3 du cours).
5. Une ou des stratégies de définition du pas sophistiquées.
6. Une méthode permettant de déterminer une bonne solution réalisable du problème primal, à partir de données collectées lors de l'exécution des méthodes optimisant la fonction duale (section 3.4.2 du cours).

1.2 Rendu

Vous devrez rendre sur Moodle, pour le **26 avril 2026**, 23h59 au plus tard :

— Un ou des fichiers source en Python implémentant les méthodes requises, et permettant de les exécuter sur les instances de test.
— Un (court) fichier au format PDF rapportant, pour le problème d'ordonnancement que vous devez traiter (travail joint avec le cours d'AONE), le travail préalable nécessaire à l'implémentation (réponses aux questions 1 et 2 de la section 2.2.2). Vous décrirez succinctement les variantes des algorithmes implémentées. Vous choisirez quelques instances de test représentatives. Pour chaque instance de test et chaque variante de méthode, vous rapporterez la valeur de la relaxation obtenue, le nombre d'itérations effectuées par l'algorithme et le temps de calcul nécessaire.

Tout rendu en retard sera pénalisé (1 point par jour).

2 Problèmes et relaxations

Vous devez implémenter la résolution du problème dual de Lagrange pour les relaxations deux problèmes donnés dans la section suivante, ainsi que les deux relaxations du problème d'ordonnancement, proposées dans la section 2.2.

2.1 Instances pour développement et débogage

La résolution du problème dual de ces deux problèmes doit être implémentée. Les contraintes sous la forme $g_i(\boldsymbol{x}) \leq 0$ doivent être dualisées, tandis que les les contraintes définissant les ensembles $\mathcal{X}$ ne doivent pas être dualisées.

$$\begin{aligned}
(P_1) : \min\ & f(\boldsymbol{x}) = x_1 - 4x_2 \\
\text{s.c.}\ & -x_1 - x_2 + 2 \leq 0 \\
& x_2 - 1 \leq 0 \\
& \boldsymbol{x} \in \mathcal{X} := \{\boldsymbol{x} : 0 \leq x_1 \leq 3, 0 \leq x_2 \leq 3\}
\end{aligned}$$

$$\begin{aligned}
(P_2) : \min\ & f(\boldsymbol{x}) = (x_1 - 2)^2 + \frac{1}{4}x_2^2 \\
\text{s.c.}\ & x_1 - \frac{7}{2}x_2 - 1 \leq 0 \\
& \boldsymbol{x} \in \mathcal{X} := \{\boldsymbol{x} : 2x_1 + 3x_2 = 4\}
\end{aligned}$$

2.2 Problème d'ordonnancement $1||\sum w_j T_j$

Vous devez implémenter la résolution des problèmes duaux définis dans les questions 1 et 2 ci-dessous.

2.2.1 The problem

We are interested in the classical single machine total weighted tardiness problem, denoted by $1||\sum w_i T_i$ in the standard scheduling notation. In this document, we use the following notations :

— I is the set of jobs.
— For all job $i \in I$, p_i denotes the processing time of i.
— $T = \{0, \ldots, \sum_{i \in I} p_i\}$ is the time horizon.
— For all job $i \in I$, $T_i = \{0, \ldots, T - p_i\}$ denotes the set of possible starting time for i.
— For all job $i \in I$ and $t \in T_i$, c_{it} is the cost of job i if it starts at time t.

We base on the so-called *time-indexed* integer linear programming model :

$$(TI) : \min \sum_{i \in I} \sum_{t \in T_i} c_{it} x_{it} \tag{1}$$

$$\text{s.t.} \sum_{t \in T_i} x_{it} = 1 \qquad \forall i \in I \tag{2}$$

$$\sum_{i \in I} \sum_{t' \in \{t - p_i + 1, \ldots, t\} \cap T_i} x_{it'} \leq 1 \qquad \forall t \in T \tag{3}$$

$$x_{it} \in \{0, 1\} \qquad \forall i \in I, t \in T_i \tag{4}$$

In this model, variable x_{it} is equal to 1 if job i starts at time t, and 0 otherwise. As a reminder, the problem is to assign a starting time to each job (Constraints (2)). In a feasible solution, at most one job at a time is processed by the machine (3). The objective is to minimize the sum of the costs over all jobs (1).

2.2.2 Lagrangian relaxations

From model (TI), one can derive two different Lagrangian relaxations. The question 2 is the continuation of the work done in the course "Algorithmes pour l'Optimisation en Nombres Entiers".

Question 1. In the first one, the *disjunctive* constraints (3) are relaxed. Thus, several jobs can be processed at a time. The resulting Lagrangian subproblem comes to a collection of independent subproblems : for each job, one has to select the starting time with the smallest Lagrangian cost. Write the corresponding Lagrangian function $\mathcal{L}^D$ and the Lagrangian dual function L^D. Write an algorithm to solve the Lagrangian subproblem.

Question 2. In the second one, the *assignment* constraints (2) are relaxed. Thus, each job can be processed zero, one, or several times. A seen in the course AONE, to reinforce the relaxation, we add the following constraint to the model :

$$\sum_{i \in I} \sum_{t \in T_i} x_{it} = n,$$

and we change the inequalities 3 to equalities. Write the corresponding $\mathcal{L}^A$ and the Lagrangian dual function L^A. Write the dynamic programming algorithm to solve the Lagrangian subproblem.

3 Conseils indicatifs

3.1 Conseils généraux

- Puisque les méthodes implémentées doivent être testées sur différentes instances, il est fortement conseillé d'écrire du code générique pour les méthodes de résolution, qui appelle du code spécifique aux instances à résoudre. Vous trouverez sur Moodle une proposition de squelette de code, qui tente un compromis entre la généricité et la simplicité, qui omet délibérément la programmation objet.
  Une approche alternative appropriée est de définir une classe représentant un programme mathématique, et implémentant différentes fonctions : calcul de la fonction objectif en un point, calcul des valeurs des fonctions des contraintes en un point, calcul de la fonction duale en un point...
- Comme toujours, commentez votre code, et utilisez des noms de variable significatifs.
- Ce travail doit se faire en binôme. Il est tout à fait autorisé de discuter avec vos collègues, consulter des livres ou des ressources en ligne afin de vous aider à le réaliser. Cependant, toute forme de plagiat sera sanctionnée.
- Pour ce travail, **l'usage d'IA génératives est interdit** (vous devez apprendre à concevoir et implémenter ces méthodes au moins une fois par vous-même).

3.2 Méthode ADS

La description classique de cette méthode omet un certain nombre de détails. Notamment :

- Lors de la première itération, la direction précédente $\boldsymbol{d}^{k-1}$ n'est pas définie.
- Lorsque la direction précédente $\boldsymbol{d}^{k-1}$ est colinéaire au sous-gradient courant $\boldsymbol{g}^k$ et de sens opposée, la direction définie $\boldsymbol{d}^k$ est le vecteur nul. Il est nécessaire de prendre en compte ce cas particulier pour assurer la correction de l'algorithme (en évitant a minima une division par zéro à l'itération suivante).

Une manière simple de gérer ces deux cas particuliers est de définir $\boldsymbol{d}^k = \boldsymbol{g}^k$ lorsqu'ils se présentent.

3.3 Méthode des plans sécants

Cette méthode nécessite la résolution d'un programme linéaire à chaque itération. Vous pouvez utiliser les librairies de votre choix dans ce but (par exemple, `Python-mip` avec le solveur `CBC`, `IBM Ilog Cplex` ou `Gurobi`).